

rs-final (grupa A)

FIPU Raspodijeljeni sustavi: *final kolokvij*

Datum: 29. 1. 2025.

Ime i prezime: _____

JMBAG: _____

Potpis: _____

VAŽNA NAPOMENA: Predajete samo jednu `zip` datoteku koja sadrži sve potrebne datoteke.

Zadatak 1 (12 bodova)

1.1 Napravite novi `FastAPI` mikroservis `FIPU_service` koji će omogućiti korisnicima da dohvate podatke o studentima i kolegijima te dodaju nove studente i kolegije u *in-memory* liste.

1.2 Implementirajte Pydantic model `student` koji sadrži sljedeće attribute:

- `JMBAG` - jedinstveni identifikator studenta
- `ime` - ime studenta
- `prezime` - prezime studenta
- `kolegiji` - lista `Kolegij`-a koje student pohađa
- `godina_studija` - godina studija studenta
- `status` - status studenta (redovan | izvanredan)

1.3 Implementirajte Pydantic model `kolegij` koji sadrži sljedeće attribute:

- `sifra` - jedinstveni identifikator kolegija
- `naziv` - naziv kolegija
- `nositelj` - nositelj kolegija
- `semestar` - semestar u kojem se kolegij održava
- `godina_odrzavanja` - godina u kojoj se kolegij održava

Možete koristiti sljedeće *dummy* podatke:

```
# FIPU_service/main.py
studenti = [
    {"JMBAG": "0303097567", "ime": "Marko", "prezime": "Marković", "kolegiji": [], "godina_studija": 1, "status": "redovan"},
    {"JMBAG": "0303011920", "ime": "Iva", "prezime": "Ivić", "kolegiji": [], "godina_studija": 4, "status": "izvanredan"},
    {"JMBAG": "0303088112", "ime": "Ana", "prezime": "Anić", "kolegiji": [], "godina_studija": 1, "status": "redovan"}
]

kolegiji = [
    {"sifra": "PROG_FIPU", "naziv": "Programiranje", "nositelj": "Tihomir Orehovački", "semestar": 1, "godina_odrzavanja": 1},
    {"sifra": "WA_FIPU", "naziv": "Web aplikacije", "nositelj": "Nikola Tanković", "semestar": 5, "godina_odrzavanja": 3},
    {"sifra": "RS_FIPU", "naziv": "Raspodijeljeni sustavi", "nositelj": "Nikola Tanković", "semestar": 1, "godina_odrzavanja": 4},
    {"sifra": "UPP_FIPU", "naziv": "Upravljanje poslovnim procesima", "nositelj": "Darko Etinger", "semestar": 5, "godina_odrzavanja": 3}
]
```

1.4 U glavnoj datoteci implementirajte sljedeće API rute:

- `GET /studenti` - vraća listu svih studenata
- `GET /studenti/{JMBAG}` - vraća podatke o studentu s traženim JMBAG-om (vratite odgovarajući `HTTPException` ako nije pronađen)
- `GET /kolegiji` - vraća listu svih kolegija
- `GET /kolegiji/{sifra}` - vraća podatke o kolegiju s traženom šifrom (vratite odgovarajući `HTTPException` ako nije pronađen)

Validirajte povratne vrijednosti za svaku od navedenih ruta kroz definirane Pydantic modele.

Zadatak 2 (15 bodova)

2.1 Definirajte dva API Routera, jedan za svaki resurs (`studenti.py`, `kolegiji.py`) i to u zasebnim datotekama unutar `routes` direktorija.

- uključite routere u glavnu aplikaciju i dodajte odgovarajuće prefikse

2.2 Nadogradite Pydantic modele `Student` i `Kolegij` sljedećim dodatnim validacijama.

- `JMBAG` mora biti duljine točno `10` znakova
- `godina_studija` mora biti u rasponu od `1` do `5`
- `status` mora biti jedan od navedenih: `redovan`, `izvanredan`
- `semestar` mora biti broj u rasponu od `1` do `8`
- `godina_odrzavanja` mora biti broj u rasponu od `1` do `5`

2.3 Implementirajte novu rutu za dodavanje studenta.

- ruta mora dodati novog studenta u listu studenata

- potrebno je definirati dodatne `Input/Output` modele za Pydantic model `Student` gdje će:
 - `Input` model sadržavati sva polja osim `JMBAG` polja, **JMBAG se mora generirati na poslužitelju**
 - definirajte **posebnu funkciju za generiranje JMBAG -a** koja će generirati JMBAG duljine `10` znakova s tim da prvih 5 znakova moraju biti `03030`.
 - nasumičnu znamenku između `0` i `9` možete generirati pomoću `random.randint(0, 9)`
 - `Output` model mora sadržavati sva polja koja definiraju studenta, uključujući i `JMBAG`
- u implementaciji rute koristite `Input` model za validaciju ulaznih podataka, a `Output` model za validaciju izlaznih podataka

2.4 Implementirajte novu rutu za dodavanje kolegija.

- ruta mora dodati novi kolegij u listu kolegija
- potrebno je definirati dodatne `Input/Output` modele za Pydantic model `Kolegij` gdje će:
 - `Input` model sadržavati sva polja osim `sifra` polja, **šifra se mora generirati na poslužitelju**
 - definirajte **posebnu funkciju za generiranje šifre** koja će generirati šifru na temelju naziva kolegija
 - ako se naziv kolegija sastoji od jedne riječi, šifra će biti prva 4 slova naziva kolegija
 - ako se naziv kolegija sastoji od više riječi, šifra će biti prvo slovo svake riječi u nazivu kolegija
 - **šifra mora biti velikim slovima**, npr. za kolegij "Operacijska istraživanja" generirana šifra mora biti: `OI_FIPU`
 - `Output` model mora sadržavati sva polja koja definiraju kolegij, uključujući i `sifra`
- u implementaciji rute koristite `Input` model za validaciju ulaznih podataka, a `Output` model za validaciju izlaznih podataka

Zadatak 3 (10 bodova)

Unutar direktorija `client` implementirajte Python skriptu u kojoj ćete simulirati slanje velikog broja konkurentnih HTTP zahtjeva prema `GET` rutama koje ste implementirali u Zadatku 1. mikroservisa `FIPU_service`.

- skripta mora koristiti `asyncio` i `aiohttp` biblioteke
- otvorite klijentsku sesiju koristeći biblioteku `aiohttp`
- definirajte korutinu `get_student` koja će slati `GET` zahtjev prema rutama `/studenti` te ju pozovite unutar `main` korutine `50` puta **konkurentno**
- definirajte korutinu `get_kolegij` koja će slati `GET` zahtjev prema rutama `/kolegiji` te ju pozovite unutar `main` korutine `30` puta **konkurentno**

Zadatak 4 (10 bodova)

Napravite novi `aiohttp` mikroservis `ISVU_service` koji će vraćati podatke o dostupnim ispitnim rokovima za određeni kolegij.

4.1 Implementirajte novu rutu `/ispitni_rokovi/{sifra}` koja će vraćati podatke o ispitnim rokovima za kolegij s traženom šifrom.

- ako šifra proslijeđena u parametru rute ne postoji u listi ispitnih rokova, vratite odgovarajući statusni kod i poruku

Podaci o ispitnim rokovima su sljedeći:

```
ispitni_rokovi = [  
    {"sifra": "PROG_FIPU", "ispitni_rok": "Ispitni rok 1", "datum": "2025-02-15"},  
    {"sifra": "PROG_FIPU", "ispitni_rok": "Ispitni rok 2", "datum": "2025-06-15"},  
    {"sifra": "PROG_FIPU", "ispitni_rok": "Ispitni rok 3", "datum": "2025-09-15"},  
    {"sifra": "WA_FIPU", "ispitni_rok": "Ispitni rok 1", "datum": "2025-02-20"},  
    {"sifra": "WA_FIPU", "ispitni_rok": "Ispitni rok 2", "datum": "2025-06-20"},  
    {"sifra": "RS_FIPU", "ispitni_rok": "Ispitni rok 1", "datum": "2025-02-25"},  
    {"sifra": "RS_FIPU", "ispitni_rok": "Ispitni rok 2", "datum": "2025-06-25"},  
    {"sifra": "UPP_FIPU", "ispitni_rok": "Ispitni rok 1", "datum": "2025-03-01"},  
    {"sifra": "UPP_FIPU", "ispitni_rok": "Ispitni rok 2", "datum": "2025-07-01"},  
]
```

4.2 Definirajte `Dockerfile` za mikroservis `ISVU_service` te ga pokrenite unutar Docker kontejnera.

- pohranite sve ovisnosti u `requirements.txt` datoteku te ju kopirajte u datotečni sustav kontejnera
- pokrenite mikroservis na portu `9000` (izložite ga prema domaćinu)
- testirajte rad mikroservisa uz pomoć HTTP klijenta po izboru

U komentare mikroservisa upišite terminal naredbe koje ste koristili **za izgradnju predloška** (`Dockerfile`) i **pokretanje kontejnera**.

Zadatak 5 (13 bodova)

5.1 Implementirajte novu rutu `GET /ispiti/{sifra}` u mikroservisu `FIPU_service` koja će koristiti `aiohttp` klijentsku sesiju za dohvat podataka o ispitnim rokovima za kolegij s traženom šifrom iz mikroservisa `ISVU_service`.

5.2 Definirajte `Dockerfile` za mikroservis `FIPU_service` te ga pokrenite unutar Docker kontejnera.

- pohranite sve ovisnosti u `requirements.txt` datoteku te ju kopirajte u datotečni sustav kontejnera
- pokrenite mikroservis na portu `8000` (izložite ga prema domaćinu)
- testirajte rad mikroservisa kroz Swagger sučelje

U komentare mikroservisa upišite terminal naredbe koje ste koristili **za izgradnju predloška** (`Dockerfile`) i **pokretanje kontejnera**.

5.3 Koristeći `Docker Compose` alat povežite mikroservise `FIPU_service` i `ISVU_service` preko zajedničke mreže te ih pokrenite zajedno, kao cjelinu.

- izmijenite implementaciju rute `GET /ispiti/{sifra}` u mikroservisu `FIPU_service` tako da koristi **ispravnu domenu** za slanje zahtjeva prema mikroservisu `ISVU_service`

Ukupni **raspodijeljeni sustav** (gdje se nalazi Compose konfiguracijska datoteka) mora biti u direktoriju: `rs-final`.